



Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en Sistemas Computacionales

Compiladores

Proyecto Final: Garabato (Logo)
Manual Técnico

Profesor:
Tecla Parra Roberto

Equipo:
Luis Ernesto Mérida de León
Héctor Said Ferreira Rodríguez
Diego García Cuanalo
Francisco Omar Gutiérrez Silva

Grupo: 5CM2

16 / 06 / 2026

Introducción

Este documento describe la arquitectura interna, las decisiones de diseño y los detalles de implementación de Garabato, un intérprete del lenguaje Logo construido en Java como proyecto final de la materia de Compiladores en ESCOM-IPN.

Logo es un lenguaje de programación de alto nivel parcialmente funcional y parcialmente estructurado, creado en 1967 por Danny Bobrow, Wally Feurzeig, Seymour Papert y Cynthia Solomon, basándose en las características del lenguaje Lisp. Fue pensado desde el inicio como una herramienta didáctica: permite enseñar la mayoría de los conceptos fundamentales de la programación (listas, archivos, entrada/salida, estructuras de control, recursividad) de una forma visual y amigable, razón por la cual es ampliamente usado para trabajar con niños y jóvenes.

Su característica más iconica es la tortuga: un cursor gráfico (originalmente un robot físico, después una representación virtual) que recibe instrucciones de movimiento y con ellas dibuja figuras sobre una pantalla. Garabato implementa esta idea desde cero en Java, incluyendo un analizador sintáctico generado con BYACC/J, una máquina virtual de pila, una tabla de símbolos, manejo de marcos de activación para funciones y procedimientos, y una interfaz gráfica construida con Java Swing/AWT.

Objetivo

El objetivo del proyecto es que, aplicando los conocimientos del curso de Compiladores, el equipo construya un intérprete completo del lenguaje Logo. Esto implica diseñar la gramática del lenguaje, generar el parser, implementar la semántica mediante una máquina de pila y exponerlo todo al usuario a través de una GUI donde pueda escribir código y ver el resultado dibujado en tiempo real.

Requerimientos Técnicos

Para compilar o ejecutar el proyecto se necesita:

- Java JDK 11 o superior: incluye el compilador (javac) y el entorno de ejecución (JRE). Con Java 11+ es suficiente para tanto compilar como ejecutar el proyecto.
- BYACC/J (Berkeley YACC para Java): herramienta necesaria únicamente si se desea regenerar el parser desde el archivo de gramática P2.y. Es un compilador de compiladores enfocado en Java, equivalente a YACC pero que genera código Java en lugar de C.
- IntelliJ IDEA (recomendado) o cualquier IDE de Java: para abrir, modificar y compilar el proyecto cómodamente. No es requerimiento estricto; el proyecto puede compilarse desde la línea de comandos.

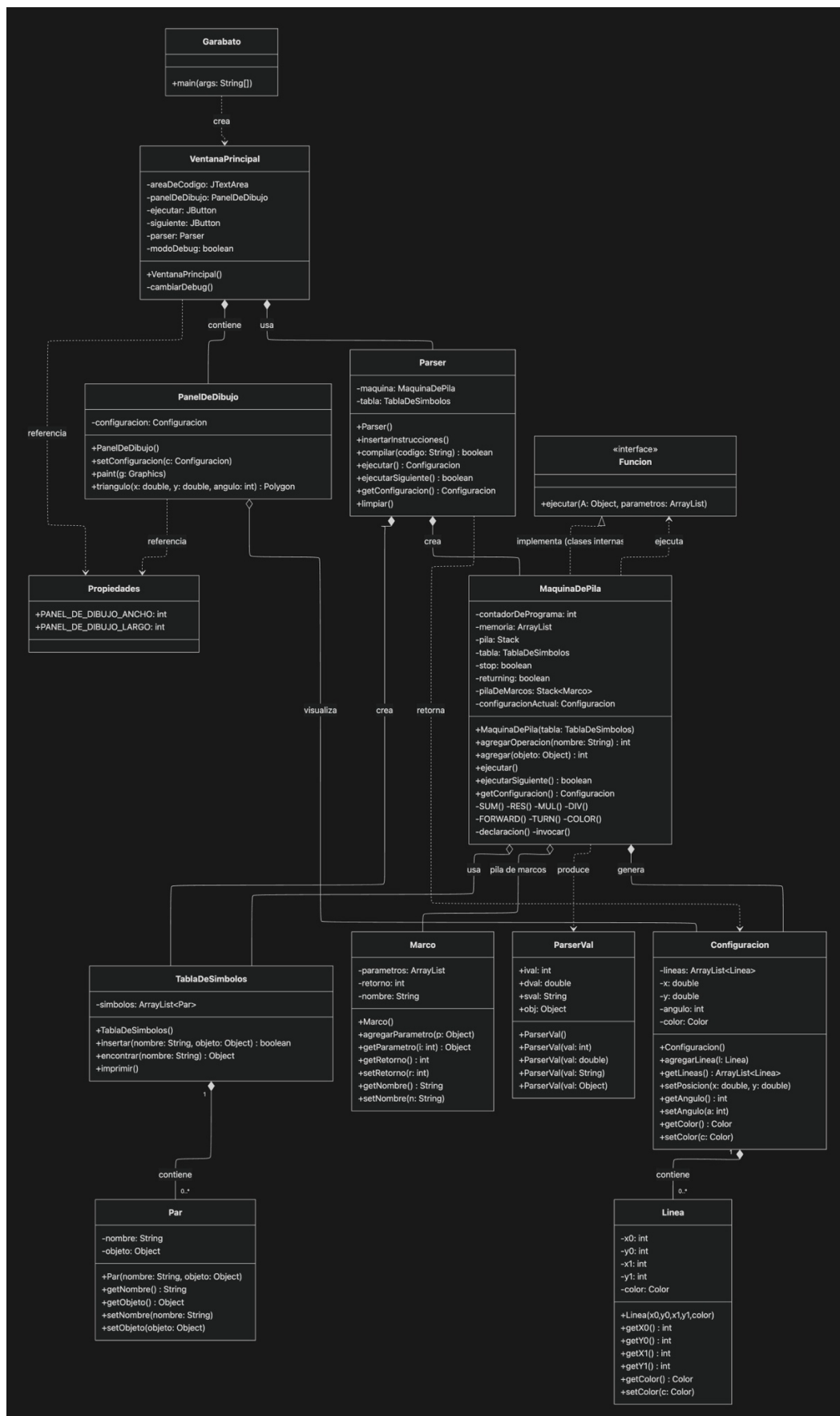
Para el usuario final, solo es necesario tener Java JDK/JRE 11 o superior instalado y ejecutar el archivo Ejecutable_Garabato.jar directamente.

Estructura del Proyecto

El código fuente está organizado en cuatro paquetes, cada uno con una responsabilidad clara:

Paquete	Archivo(s)	Responsabilidad
Garabato	Garabato.java	Clase principal. Arranca la aplicación creando la ventana principal.
Compilador	P2.y / Parser.java	Gramática YACC y parser generado por BYACC/J.
Compilador	MaquinaDePila.java	Núcleo del intérprete. Ejecuta instrucciones, evalúa expresiones y llama funciones.
Compilador	TablaDeSimbolos.java	Almacena variables y funciones declaradas por el usuario.
Compilador	Marco.java	Representa un marco de activación para manejo de funciones y sus parámetros.
Compilador	Par.java	Estructura auxiliar nombre↔objeto usada por la tabla de símbolos.
Compilador	Funcion.java	Interfaz que define el contrato de ejecución para los comandos Logo.
Compilador	ParserVal.java	Clase generada por BYACC que contiene el valor semántico de cada token.
Configuracion	Configuracion.java	Estado actual de la tortuga: posición, ángulo, color y lista de líneas dibujadas.
Configuracion	Linea.java	Representa un segmento de línea con sus coordenadas y color.
Vista	VentanaPrincipal.java	GUI principal: caja de texto, botones y panel de dibujo.
Vista	PanelDeDibujo.java	Componente Swing que pinta las líneas y el triángulo-tortuga en pantalla.
Vista	Propiedades.java	Constantes globales: dimensiones del panel de dibujo.

Arquitectura General



El flujo de ejecución de Garabato sigue el pipeline clásico de un intérprete:

Etapas	#	Descripción
Entrada	1	El usuario escribe código Logo en la caja de texto de la GUI.
Análisis	2	El parser (generado por BYACC/J a partir de P2.y) analiza léxica y sintácticamente el texto y construye la representación interna del programa en memoria.
Semántica	3	Durante el análisis, se generan instrucciones para la máquina de pila. Cada acción semántica en la gramática llama a métodos de MaquinaDePila para agregar operaciones o datos a la memoria del programa.
Ejecución	4	La máquina de pila recorre su memoria instrucción por instrucción. Cada operación puede modificar la pila de datos, la tabla de símbolos o la configuración de la tortuga.
Visualización	5	Cada vez que la tortuga avanza, se agrega un objeto Linea a la lista interna de Configuración. El PanelDeDibujo repinta el lienzo dibujando todas esas líneas acumuladas.

Componentes Internos

Parser y Gramática (P2.y)

```
%token IF
%token ELSE
%token WHILE
%token FOR
%token COMP
%token DIFERENTES
%token MAY
%token MEN
%token MAYI
%token MENI
%token FNCT
%token NUMBER
%token VAR
%token AND
%token OR
%token FUNC
%token RETURN
%token PARAMETRO
%token PROC
%right '='
%left '+' '-'
%left '*' '/'
%left ';'
%left COMP
%left DIFERENTES
%left MAY
%left MAYI
%left MEN
%left MENI
%left '!'
%left AND
%left OR
%right RETURN
%%

list:
```

```

| list '\n'
| list linea '\n'
;

linea: exp ';' {$$ = $1;}
| stmt {$$ = $1;}
| linea exp ';' {$$ = $1;}
| linea stmt {$$ = $1;}
;

exp: VAR {
    $$ = new
ParserVal(maquina.agregarOperacion("varPush_Eval"));
    maquina.agregar($1.sval);
}
| '-' exp {
    $$ = new
ParserVal(maquina.agregarOperacion("negativo"));
}
| NUMBER {
    $$ = new
ParserVal(maquina.agregarOperacion("constPush"));
    maquina.agregar($1.dval);
}
| VAR '=' exp {
    $$ = new ParserVal($3.ival);
    maquina.agregarOperacion("varPush");
    maquina.agregar($1.sval);
    maquina.agregarOperacion("asignar");
    maquina.agregarOperacion("varPush_Eval");
    maquina.agregar($1.sval);
}
| exp '*' exp {
    $$ = new ParserVal($1.ival);
    maquina.agregarOperacion("MUL");
}
| exp '/' exp {
    $$ = new ParserVal($1.ival);
    maquina.agregarOperacion("DIV");
}
| exp '+' exp {
    $$ = new ParserVal($1.ival);
    maquina.agregarOperacion("SUM");
}
| exp '-' exp {
    $$ = new ParserVal($1.ival);
    maquina.agregarOperacion("RES");
}
| '(' exp ')' {
    $$ = new ParserVal($2.ival);
}
| exp COMP exp {
    maquina.agregarOperacion("EQ");
    $$ = $1;
}
| exp DIFERENTES exp {
    maquina.agregarOperacion("NE");
    $$ = $1;
}
| exp MEN exp {
    maquina.agregarOperacion("LE");
    $$ = $1;
}
| exp MENI exp {
    maquina.agregarOperacion("LQ");
}

```

```

        $$ = $1;
    }
    | exp MAY exp {
        maquina.agregarOperacion("GR");
        $$ = $1;
    }
    | exp MAYI exp {
        maquina.agregarOperacion("GE");
        $$ = $1;
    }
    | exp AND exp {
        maquina.agregarOperacion("AND");
        $$ = $1;
    }
    | exp OR exp {
        maquina.agregarOperacion("OR");
        $$ = $1;
    }
    | '!' exp {
        maquina.agregarOperacion("NOT");
        $$ = $2;
    }
    | RETURN exp { $$ = $2; maquina.agregarOperacion("_return");
}

    | PARAMETRO { $$ = new
ParserVal(maquina.agregarOperacion("push_parametro"));
maquina.agregar((int)$1.ival); }

    | nombreProc '(' arglist ')' { $$ = new
ParserVal(maquina.agregarOperacionEn("invocar", ($1.ival)));
maquina.agregar(null); } //instrucciones tiene la estructura necesaria
para la lista de argumentos
    ;

    arglist:
    | exp { $$ = $1; maquina.agregar("Limite"); }
    | arglist ',' exp { $$ = $1; maquina.agregar("Limite"); }
    ;

    nop: { $$ = new ParserVal(maquina.agregarOperacion("nop")); }
    ;

    stmt: if '(' exp stop ')' '{' linea stop '}' ELSE '{' linea stop '}'
{
    $$ = $1;
    maquina.agregar($7.ival, $1.ival + 1);
    maquina.agregar($12.ival, $1.ival + 2);
    maquina.agregar(maquina.numeroDeElementos() -
1, $1.ival + 3);
}
    | if '(' exp stop ')' '{' linea stop '}' nop stop{
    $$ = $1;
    maquina.agregar($7.ival, $1.ival + 1);
    maquina.agregar($10.ival, $1.ival + 2);
    maquina.agregar(maquina.numeroDeElementos() -
1, $1.ival + 3);
}
    | while '(' exp stop ')' '{' linea stop '}' stop{
    $$ = $1;
    maquina.agregar($7.ival, $1.ival + 1);
    maquina.agregar($10.ival, $1.ival + 2);
}
    | for '(' instrucciones stop ';' exp stop ';' instrucciones
stop ')' '{' linea stop '}' stop{

```

```

        $$ = $1;
        maquina.agregar($6.ival, $1.ival + 1);
        maquina.agregar($9.ival, $1.ival + 2);
        maquina.agregar($13.ival, $1.ival + 3);
        maquina.agregar($16.ival, $1.ival + 4);
    }
    | funcion nombreProc '(' ')' '{' ' linea null '}'
    | procedimiento nombreProc '(' ')' '{' ' linea null '}'
    | instruccion '[' arglist ']' ';' {
        $$ = new ParserVal($1.ival);
        maquina.agregar(null);
    }
    ;
    instruccion: FNCT {
        $$ = new
ParserVal(maquina.agregar((Funcion)($1.obj)));
    }
    ;

    procedimiento: PROC { maquina.agregarOperacion("declaracion"); }
    ;
    funcion: FUNC { maquina.agregarOperacion("declaracion"); }
    ;

    nombreProc: VAR { $$ = new ParserVal(maquina.agregar($1.sval)); }
    ;

    null: {maquina.agregar(null);}
    ;

    stop: { $$ = new ParserVal(maquina.agregarOperacion("stop")); }
    ;

    if: IF {
        $$ = new
ParserVal(maquina.agregarOperacion("IF_ELSE"));
        maquina.agregarOperacion("stop");//then
        maquina.agregarOperacion("stop");//else
        maquina.agregarOperacion("stop");//siguiente comando
    }
    ;

    while: WHILE {
        $$ = new
ParserVal(maquina.agregarOperacion("WHILE"));
        maquina.agregarOperacion("stop");//cuerpo
        maquina.agregarOperacion("stop");//final
    }
    ;

    for : FOR {
        $$ = new ParserVal(maquina.agregarOperacion("FOR"));
        maquina.agregarOperacion("stop");//condicion
        maquina.agregarOperacion("stop");//instrucción final
        maquina.agregarOperacion("stop");//cuerpo
        maquina.agregarOperacion("stop");//final
    }

    instrucciones: { $$ = new
ParserVal(maquina.agregarOperacion("nop")); }
    | exp { $$ = $1; }
    | instrucciones ',' exp { $$ = $1; }
    ;

%%

```


La gramática del lenguaje está definida en el archivo P2.y usando la sintaxis de YACC. Al compilarlo con BYACC/J se genera automáticamente el archivo Parser.java, que contiene el analizador léxico (yylex) y sintáctico (yyvsparse) del lenguaje.

Los tokens reconocidos por el lenguaje incluyen: IF, ELSE, WHILE, FOR, COMP (==), DIFERENTES (!=), MAY (>), MEN (<), MAYI (>=), MENI (<=), FNCT (para funciones Logo), NUMBER, VAR, AND, OR, FUNC, RETURN, PARAMETRO, PROC, entre otros. Los operadores aritméticos (+, -, *, /) tienen asociatividad izquierda y prioridad estándar.

Las reglas gramaticales más relevantes son:

- **list / línea:** punto de entrada del programa. Un programa es una secuencia de líneas separadas por saltos de línea.
- **exp:** maneja expresiones aritméticas, asignaciones de variables, comparaciones lógicas, negación, paréntesis, acceso a parámetros (\$n), llamadas a funciones/procedimientos y el valor de retorno.
- **stmt:** engloba las estructuras de control: if-else, while, for. Cada una genera instrucciones de salto en la máquina de pila con punteros calculados en tiempo de parseo.
- **instruccion:** cubre los comandos Logo de dibujo (FORWARD, TURN, COLOR, PenUP, PenDOWN) instanciados como objetos que implementan la interfaz Funcion.
- **funcion / procedimiento / nombreProc:** permiten declarar funciones y procedimientos de usuario que quedan registrados en la tabla de símbolos apuntando a su posición en memoria.

Máquina de Pila (MaquinaDePila.java)

```
public void ejecutarInstruccion(int indice){
    //System.out.println("Ejecutando: " + indice);
    try{
        Object objetoLeido = memoria.get(indice);
        if(objetoLeido instanceof Method){
            Method metodo = (Method)objetoLeido;
            metodo.invoke(this, null);
        }
        if(objetoLeido instanceof Funcion){
            ArrayList parametros = new ArrayList();
            Funcion funcion = (Funcion)objetoLeido;
            contadorDePrograma++;
            while(memoria.get(contadorDePrograma) != null){ //Aquí también usamos null como delimitador. Aquí se agregan
                if(memoria.get(contadorDePrograma) instanceof String){
                    if(((String)(memoria.get(contadorDePrograma))).equals("Limite")){
                        Object parametro = pila.pop();
                        parametros.add(parametro);
                        contadorDePrograma++;
                    }
                }
                else{
                    ejecutarInstruccion(contadorDePrograma);
                }
            }
            funcion.ejecutar(configuracionActual, parametros);
        }
        contadorDePrograma++;
    }
    catch(Exception e){}
```

```

private void declaracion(){
    tabla.insertar((String)memoria.get(++contadorDePrograma), ++contadorDePrograma); //Apuntamos a la primera instrucción
    int invocados = 0;
    while(memoria.get(contadorDePrograma) != null || invocados != 0){ //Llevamos cp hasta la siguiente instrucción después
        if( memoria.get(contadorDePrograma) instanceof Method)
            if(((Method)memoria.get(contadorDePrograma)).getName().equals("invocar"))
                invocados++;
        if( memoria.get(contadorDePrograma) instanceof Funcion)
            invocados++;
        if(memoria.get(contadorDePrograma) == null)
            invocados--;
        contadorDePrograma++;
    }
}

```

```

private void invocar(){
    Marco marco = new Marco();
    String nombre = (String)memoria.get(++contadorDePrograma);
    marco.setNombre(nombre);
    contadorDePrograma++;
    while(memoria.get(contadorDePrograma) != null){ //Aquí también usamos null como delimitador. Aquí se agregan los parámetros
        if(memoria.get(contadorDePrograma) instanceof String){
            if(((String)(memoria.get(contadorDePrograma))).equals("Limite")){
                Object parametro = pila.pop();
                marco.agregarParametro(parametro);
                contadorDePrograma++;
            }
        }
        else{
            ejecutarInstruccion(contadorDePrograma);
        }
    }
    marco.setRetorno(contadorDePrograma);
    pilaDeMarcos.add(marco);
    ejecutarFuncion((int)tabla.encontrar(nombre)); //VAMOS AQUI*****
}

```

Es el componente central del intérprete. Funciona con dos estructuras principales:

- **memoria (ArrayList):** lista lineal donde el parser deposita objetos durante el análisis. Puede contener referencias a métodos de Java (instancias de Method, invocados por reflexión), objetos que implementan Funcion (los comandos Logo), valores literales (doubles, strings) o null como delimitador de bloques.
- **pila (Stack):** pila de operandos para evaluar expresiones. Las operaciones aritméticas y lógicas sacan sus argumentos de aquí y dejan el resultado.

Las operaciones que la máquina soporta internamente incluyen:

- Aritméticas: SUM, RES, MUL, DIV y negativo.
- Pila de datos: constPush (empuja un literal), varPush (empuja el nombre de una variable), varPush_Eval (empuja el valor de una variable desde la tabla de símbolos), asignar.
- Comparaciones: EQ (==), NE (!=), LE (<), LQ (<=), GR (>), GE (>=).
- Lógicas: AND, OR, NOT.
- Control de flujo: WHILE, FOR, IF_ELSE, stop, nop.
- Funciones y procedimientos: declaracion (registra en tabla de símbolos), invocar (crea un Marco, carga parámetros, ejecuta el cuerpo), push_parametro (carga el parámetro \$n del marco activo), _return.

- Comandos Logo: Avanzar, Girar, CambiarColor, SubirPincel, BajarPincel — implementados como clases internas estáticas que implementan la interfaz Funcion.

Los métodos públicos de ejecución son tres:

- **ejecutar():** recorre la memoria de inicio a fin ejecutando todas las instrucciones. Lo usa el botón Run.
- **ejecutarSiguiente():** ejecuta únicamente la instrucción apuntada por el contador de programa y avanza. Lo usa el modo Debug paso a paso.
- **ejecutar(int indice):** ejecuta desde un índice dado hasta encontrar una señal de stop. Lo usan internamente las estructuras de control y las llamadas a funciones.

Tabla de Símbolos (TablaDeSimbolos.java)

Implementada como un ArrayList de objetos Par (nombre + objeto). Ofrece tres operaciones:

- **insertar(nombre, objeto):** si el nombre ya existe, actualiza su valor; si no, agrega un nuevo Par. Sirve tanto para variables de usuario como para registrar funciones/procedimientos por su nombre.
- **encontrar(nombre):** recorre la lista buscando el Par cuyo nombre coincida y retorna el objeto asociado. Retorna null si no lo encuentra.
- **imprimir():** método de depuración que imprime en consola todos los símbolos registrados.

Marcos de Activación (Marco.java)

Cada vez que se invoca una función o procedimiento, la máquina crea un objeto Marco y lo empuja a la pilaDeMarcos. El marco guarda:

- **nombre:** identificador de la función invocada.
- **parametros (ArrayList):** lista de valores pasados como argumentos, que se acceden con \$1, \$2, etc. desde dentro del cuerpo.
- **retorno (int):** posición en memoria a la que debe volver el contador de programa al terminar la ejecución de la función.

Al terminar la ejecución de la función, el marco se saca de la pila y el contador de programa regresa al punto de retorno almacenado. Este mecanismo permite llamadas recursivas, ya que cada nivel de recursión tiene su propio marco con sus propios parámetros.

Configuración y Modelo Gráfico

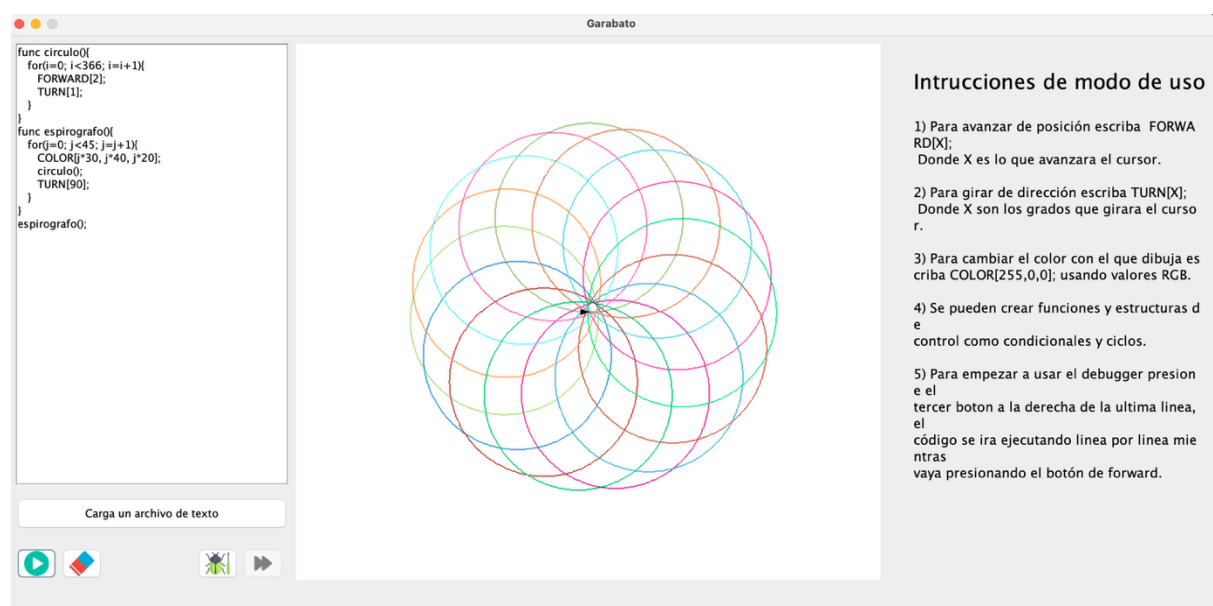
El estado de la tortuga se centraliza en un objeto Configuracion que mantiene:

- **x, y (double):** posición actual de la tortuga en el plano. El origen (0,0) corresponde al centro del panel.
- **angulo (int):** ángulo de orientación en grados. Se usa para calcular las componentes (cos, sin) del desplazamiento en cada FORWARD.

- **color (Color):** color del trazo activo. SubirPincel lo pone en Color.GRAY (efecto de no dibujar) y BajarPincel lo regresa a Color.BLACK. COLOR[R,G,B] lo cambia explícitamente.
- **lineas (ArrayList):** acumulación de todos los segmentos de línea generados. Es la "memoria visual" del dibujo.

Cada vez que la clase Avanzar ejecuta un FORWARD, calcula los puntos extremos de la línea usando trigonometría básica (cos/sin sobre el ángulo actual) y agrega un objeto Linea con esas coordenadas y el color activo.

Interfaz Gráfica (Vista)



La GUI está construida con Java Swing/AWT y consta de los siguientes componentes principales:

- **VentanaPrincipal.java:** ensambla todos los elementos visuales. Contiene el área de texto para el código, los cuatro botones (Run, Clear, Debug, Siguiente) y el panel de dibujo. También gestiona la lógica de los botones: al presionar Run instancia el parser y llama a ejecutar(); al presionar Siguiente llama a ejecutarSiguiente() de la máquina.
- **PanelDeDibujo.java:** extiende JPanel y sobrescribe el método paint(Graphics g). En cada repintado itera sobre la lista de líneas de Configuración y las dibuja con g.drawLine(). Además, calcula y dibuja el triángulo relleno que representa la posición y dirección actual de la tortuga, usando cálculos trigonométricos para los tres vértices del polígono.
- **Propiedades.java:** clase de constantes que define el ancho y alto del panel de dibujo. El panel usa esas dimensiones para trasladar las coordenadas lógicas de la tortuga (origen en el centro) a coordenadas de pantalla (origen en la esquina superior izquierda de Swing).

Gramática Completa

A continuación se listan las expresiones regulares y reglas sintácticas que definen el lenguaje reconocido por Garabato:

Comandos Logo (instrucciones de dibujo)

```
FORWARD[<expresion>;  
TURN[<expresion>;  
COLOR[<expr>, <expr>, <expr>;  
PenUP[;  
PenDOWN[;
```

Estructuras de control

```
for ( <inicio>; <condicion>; <incremento> ) { <instrucciones> }  
while ( <condicion> ) { <instrucciones> }  
if ( <condicion> ) { <instrucciones> } else { <instrucciones> }
```

Funciones y procedimientos

```
proc <nombre>() { <instrucciones> }  
func <nombre>() { <instrucciones> return <valor>; }  
  
// Acceso a parámetros dentro del cuerpo:  
$1, $2, $3, ...  
  
// Llamada con argumentos separados por coma:  
<nombre>(<arg1>, <arg2>;
```

Operadores soportados

```
Aritméticos:  +   -   *   /   (negación unaria -)  
Comparación:  ==  !=  <  <=  >  >=  
Lógicos:      &&  ||  !  
Asignación:   variable = <expresion>
```

Descripción de Clases Clave

MaquinaDePila

A continuación se explica el comportamiento de los métodos más relevantes de la clase:

declaracion(): cuando el parser encuentra una función o procedimiento, llama a este método. Inserta en la tabla de símbolos el nombre de la función apuntando a la siguiente instrucción de la memoria (la primera línea de su cuerpo). Después avanza el contador de programa hasta saltar todo el cuerpo de la función, usando un contador de invocaciones anidadas para manejar funciones que contienen otras funciones.

invocar(): cuando el intérprete encuentra una llamada a función, crea un Marco nuevo, lee el nombre de la función desde memoria, carga los parámetros que están en la pila al marco, almacena la posición de retorno y luego llama a ejecutarFuncion() para saltar al cuerpo de la función.

ejecutarInstruccion(int indice): método central del ciclo de ejecución. Lee el objeto almacenado en la posición de memoria indicada y toma una decisión: si es un Method (operación aritmética, de control, etc.) lo invoca por reflexión usando metodo.invoke(this, null); si es una instancia de Funcion (un comando Logo), recoge sus parámetros y llama a funcion.ejecutar(configuracionActual, parametros).

Avanzar.ejecutar(): implementación del comando FORWARD. Obtiene el ángulo y posición actual de la Configuracion, calcula el nuevo punto usando Math.cos y Math.sin sobre el ángulo en radianes, actualiza la posición y agrega la nueva Linea a la lista de segmentos dibujados.

SubirPincel / BajarPincel: simulan el efecto del lápiz levantado o bajado cambiando el color activo a Color.GRAY (invisible sobre el fondo blanco) o Color.BLACK respectivamente. Es una solución simple y efectiva que no requiere una bandera adicional de estado.

Par y TablaDeSimbolos

La clase Par es una estructura genérica de par (clave, valor): guarda un String nombre y un Object objeto, con sus respectivos getters y setters. La TablaDeSimbolos los almacena en un ArrayList y los busca de forma lineal. Esta implementación es directa y suficiente para el alcance del proyecto; no requiere hashing porque el número de símbolos definidos en un programa Logo típico es pequeño.

Linea y Configuracion

Linea encapsula un segmento: coordenadas $(x_0, y_0) \rightarrow (x_1, y_1)$ y un Color. Configuracion funciona como el "estado global" de la tortuga: posición, ángulo, color activo y la lista creciente de Lineas. Es el objeto que se pasa como argumento a cada Funcion Logo cuando se ejecuta, lo que permite que los comandos modifiquen el estado del dibujo.

Código Representativo

Los siguientes fragmentos ilustran los mecanismos internos más importantes del intérprete. Para ver el código completo consultar los archivos fuente del proyecto.

Ciclo principal de ejecución

```
public void ejecutarInstruccion(int indice) {
    try {
        Object obj = memoria.get(indice);
        if (obj instanceof Method) {
            // Operación interna: aritmética, control de flujo, etc.
            ((Method) obj).invoke(this, null);
        }
        if (obj instanceof Funcion) {
            // Comando Logo: FORWARD, TURN, COLOR, etc.
            ArrayList params = new ArrayList();
            Funcion f = (Funcion) obj;
            contadorDePrograma++;
            // ...cargar parámetros desde la pila...
            f.ejecutar(configuracionActual, params);
        }
        contadorDePrograma++;
    } catch (Exception e) {}
}
```

Comando FORWARD (clase Avanzar)

```
public static class Avanzar implements Funcion {
    public void ejecutar(Object A, ArrayList parametros) {
        Configuracion cfg = (Configuracion) A;
        int angulo = cfg.getAngulo();
        double x0 = cfg.getX(), y0 = cfg.getY();
        double dist = (double) parametros.get(0);
        double x1 = x0 + Math.cos(Math.toRadians(angulo)) * dist;
        double y1 = y0 + Math.sin(Math.toRadians(angulo)) * dist;
        cfg.setPosicion(x1, y1);
        cfg.agregarLinea(new Linea((int)x0, (int)y0, (int)x1, (int)y1,
                                   cfg.getColor()));
    }
}
```

PenUP / PenDOWN: lápiz virtual

```
// Levantar el lápiz: líneas grises = invisibles sobre fondo blanco
public static class SubirPincel implements Funcion {
    public void ejecutar(Object A, ArrayList p) {
        ((Configuracion) A).setColor(Color.GRAY);
    }
}

// Bajar el lápiz: retomar el trazo negro
public static class BajarPincel implements Funcion {
    public void ejecutar(Object A, ArrayList p) {
        ((Configuracion) A).setColor(Color.BLACK);
    }
}
```

Ejemplos de Programas Logo

El proyecto incluye un archivo de ejemplos (Ejemplos de Entradas.txt) con programas que demuestran las capacidades del intérprete. A continuación se muestran algunos representativos:

Cuadrado con colores (multi-color)

```
PenUP[];
FORWARD[100];
TURN[90];
PenDOWN[];
COLOR[255,0,0];
FORWARD[100]; TURN[90];
COLOR[0,255,0];
FORWARD[100]; TURN[90];
COLOR[0,0,255];
FORWARD[100]; TURN[90];
```

Polígonos regulares con ciclo for

Cualquier polígono regular de n lados se dibuja con la fórmula: ángulo de giro = $360 / n$.

```
// Triángulo (n=3, ángulo=120)
for(i = 0; i < 3; i = i + 1) {
    FORWARD[100]; TURN[120];
}

// Hexágono (n=6, ángulo=60)
for(i = 0; i < 6; i = i + 1) {
    FORWARD[100]; TURN[60];
}
```

Curva de Koch (fractal recursivo)

```
proc koch() {
    if($1 == 0) {
        FORWARD[$2];
    } else {
        koch($1-1, $2/3);
        TURN[60];
        koch($1-1, $2/3);
        TURN[-120];
        koch($1-1, $2/3);
        TURN[60];
        koch($1-1, $2/3);
    }
}

// Copo de nieve: tres curvas de nivel 4
koch(4, 300); TURN[-120];
koch(4, 300); TURN[-120];
koch(4, 300);
```


Árbol fractal recursivo

```
proc arbol() {  
    if($1 > 5) {  
        FORWARD[$1];  
        TURN[20];    arbol($1-5);  
        TURN[320];   arbol($1-5);  
        TURN[20];    FORWARD[(-1)*($1)];  
    }  
}  
  
TURN[-90];  
FORWARD[200];  
COLOR[0, 143, 57];  
TURN[180];  
arbol(65);
```

Cómo Extender el Proyecto

En caso de querer agregar nuevas funcionalidades al intérprete, los puntos de modificación principales son los siguientes:

1. Agregar un nuevo comando Logo (por ejemplo CIRCLE[r]): crear una clase estática en MaquinaDePila.java que implemente la interfaz Funcion, definir el método ejecutar() con la lógica geométrica deseada y registrarla en P2.y como un nuevo token y una nueva regla gramatical.
2. Agregar un nuevo operador o estructura de control: añadir el token en la sección de declaraciones de P2.y, definir su regla sintáctica con las acciones semánticas correspondientes (llamadas a agregarOperacion) y, si la operación es interna, agregar el método privado en MaquinaDePila.java.
3. Recompilar el parser: tras modificar P2.y, ejecutar byacc -Jclass Parser -Jpackage Compilador P2.y para generar un nuevo Parser.java.
4. Agregar color de fondo o múltiples colores de lápiz: modificar Configuracion.java para manejar un estado de lápiz más rico (activo/inactivo + color elegido) en lugar del truco actual de Color.GRAY para PenUP.

Notas de Implementación

- El truco de PenUP/PenDOWN usando Color.GRAY funciona porque el fondo del panel es blanco: las líneas grises se dibujan pero son visualmente indistinguibles. Si el fondo cambiara, habría que agregar una bandera booleana en Configuracion.
- La búsqueda en TablaDeSimbolos es lineal $O(n)$. Para proyectos con muchos símbolos conviene sustituirla por un HashMap.
- El parser genera código de tres direcciones implícito para las estructuras de control (if, while, for) usando posiciones de memoria como "etiquetas". Esto se hace calculando offsets relativos en las acciones semánticas de la gramática.
- La reflexión de Java (Method.invoke) se usa para llamar dinámicamente a las operaciones internas de la máquina. Esto hace el código flexible pero puede tener overhead en programas muy largos.
- El nombre original del proyecto era ProyectoLogos / Logos. Fue renombrado a Garabato para el entregable final de la materia en 2026.